Appendix

Source Code Listings

The MATLAB code below implements the image color-model analysis, stain normalization, compression, and watermarking procedures referenced in this research, organized by phase and part.

Phase 1 – Part 1

```
% ========================================================================
% Phase 1: Colour Models and Stain Analysis (BreaKHis_v1)
% Implemented from scratch without built-in library calls
% ========================================================================

function phase1_stain_analysis()
    desktop_path = fullfile(getenv('USERPROFILE'), 'Desktop');
    input_folder = fullfile(desktop_path, 'BreaKHis_v1');
    output_folder = fullfile(desktop_path, 'Phase1_Channel_Outputs');

    if ~exist(output_folder, 'dir'), mkdir(output_folder); end

    % 1. Step 1: Verification
    fprintf('=== Step 1: Verifying Round-Trip Accuracy ===\n');
    test_rgb = rand(32, 32, 3);
    err = round_trip_error(test_rgb);
    fprintf('CMY Max Abs Error: %e\n', err.cmy);
    fprintf('HSI Max Abs Error: %e\n', err.hsi);
    fprintf('LAB Max Abs Error: %e\n\n', err.lab);

    % 2. Step 2: Processing Images
    image_files = dir(fullfile(input_folder, '**', '*.png'));
    if isempty(image_files)
        image_files = dir(fullfile(input_folder, '**', '*.jpg'));
    end

    if isempty(image_files)
        fprintf('Warning: No images found in BreaKHis_v1 directory.\n');
        return;
    end

    fprintf('=== Step 2: Processing Images & Separating Channels ===\n');
    num_to_process = min(10, length(image_files));

    for k = 1:num_to_process
        full_path = fullfile(image_files(k).folder, image_files(k).name);
        img_rgb = im2double(imread(full_path));
        [~, fname, ~] = fileparts(image_files(k).name);

        % --- HSI ---
        hsi = rgb_to_hsi(img_rgb);
        imwrite(mat2gray(hsi(:,:,1)), fullfile(output_folder, [fname '_HSI_H_Hue.png']));
        imwrite(hsi(:,:,2), fullfile(output_folder, [fname '_HSI_S_Sat.png']));
        imwrite(hsi(:,:,3), fullfile(output_folder, [fname '_HSI_I_Int.png']));

        % --- CMY ---
```

```
        cmy = rgb_to_cmy(img_rgb);
        imwrite(cmy(:,:,1), fullfile(output_folder, [fname '_CMY_C.png']));
        imwrite(cmy(:,:,2), fullfile(output_folder, [fname '_CMY_M.png']));
        imwrite(cmy(:,:,3), fullfile(output_folder, [fname '_CMY_Y.png']));

        % --- CIE L*a*b* ---
        lab = rgb_to_lab(img_rgb);
        imwrite(lab(:,:,1)/100, fullfile(output_folder, [fname '_LAB_L.png']));
        imwrite(mat2gray(lab(:,:,2)), fullfile(output_folder, [fname '_LAB_a.png']));
        imwrite(mat2gray(lab(:,:,3)), fullfile(output_folder, [fname '_LAB_b.png']));
    end

    fprintf('Successfully processed and saved channels to: %s\n', output_folder);
end

% ---------------------------------------------------------------------- Round-Trip Check
function err = round_trip_error(rgb)
    err.cmy = max(abs(rgb(:) - reshape(cmy_to_rgb(rgb_to_cmy(rgb)), [], 1)));
    err.hsi = max(abs(rgb(:) - reshape(hsi_to_rgb(rgb_to_hsi(rgb)), [], 1)));
    err.lab = max(abs(rgb(:) - reshape(lab_to_rgb(rgb_to_lab(rgb)), [], 1)));
end

% ---------------------------------------------------------------------- CMY
function cmy = rgb_to_cmy(rgb), cmy = 1.0 - rgb; end
function rgb = cmy_to_rgb(cmy), rgb = 1.0 - cmy; end

% ---------------------------------------------------------------------- HSI
function hsi = rgb_to_hsi(rgb)
    EPS = 1e-8; r = rgb(:,:,1); g = rgb(:,:,2); b = rgb(:,:,3);
    num = 0.5 * ((r - g) + (r - b));
    den = sqrt((r - g).^2 + (r - b).*(g - b)) + EPS;
    theta = acos(min(max(num ./ den, -1.0), 1.0));
    h = theta; h(b > g) = 2 * pi - theta(b > g); h = h / (2 * pi);
    s = 1.0 - 3.0 * min(min(r, g), b) ./ (r + g + b + EPS);
    i = (r + g + b) / 3.0;
    hsi = cat(3, h, s, i);
end

function rgb = hsi_to_rgb(hsi)
    EPS = 1e-8;
    h = hsi(:,:,1) * 2 * pi; s = hsi(:,:,2); i = hsi(:,:,3);
    [rows, cols, ~] = size(hsi);
    out = zeros(rows, cols, 3);

    m1 = h < (2 * pi / 3);
    hh = h(m1);
    b1 = i(m1) .* (1 - s(m1));
    r1 = i(m1) .* (1 + s(m1) .* cos(hh) ./ (cos(pi / 3 - hh) + EPS));
    g1 = 3 * i(m1) - (r1 + b1);

    m2 = (h >= (2 * pi / 3)) & (h < (4 * pi / 3));
    hh = h(m2) - 2 * pi / 3;
    r2 = i(m2) .* (1 - s(m2));
    g2 = i(m2) .* (1 + s(m2) .* cos(hh) ./ (cos(pi / 3 - hh) + EPS));
    b2 = 3 * i(m2) - (r2 + g2);

    m3 = h >= (4 * pi / 3);
    hh = h(m3) - 4 * pi / 3;
    g3 = i(m3) .* (1 - s(m3));
    b3 = i(m3) .* (1 + s(m3) .* cos(hh) ./ (cos(pi / 3 - hh) + EPS));
```

```
    r3 = 3 * i(m3) - (g3 + b3);

    r_chan = zeros(rows, cols); g_chan = zeros(rows, cols); b_chan = zeros(rows, cols);
    r_chan(m1) = r1; g_chan(m1) = g1; b_chan(m1) = b1;
    r_chan(m2) = r2; g_chan(m2) = g2; b_chan(m2) = b2;
    r_chan(m3) = r3; g_chan(m3) = g3; b_chan(m3) = b3;

    out(:,:,1) = r_chan; out(:,:,2) = g_chan; out(:,:,3) = b_chan;
    rgb = min(max(out, 0.0), 1.0);
end

% ------------------------------------------------------------------ CIE Lab & XYZ
function M = get_M_RGB2XYZ()
    M = [0.4124564, 0.3575761, 0.1804375; ...
         0.2126729, 0.7151522, 0.0721750; ...
         0.0193339, 0.1191920, 0.9503041];
end

function lin = srgb_to_linear(c)
    lin = zeros(size(c)); mask = c <= 0.04045;
    lin(mask) = c(mask) / 12.92; lin(~mask) = ((c(~mask) + 0.055) / 1.055) .^ 2.4;
end

function srgb = linear_to_srgb(c)
    c = max(c, 0); srgb = zeros(size(c)); mask = c <= 0.0031308;
    srgb(mask) = 12.92 * c(mask); srgb(~mask) = 1.055 * (c(~mask) .^ (1 / 2.4)) - 0.055;
end

function xyz = rgb_to_xyz(rgb)
    lin = srgb_to_linear(rgb); M = get_M_RGB2XYZ(); [r, c, ch] = size(lin);
    xyz = reshape(reshape(lin, [], ch) * M', r, c, ch);
end

function rgb = xyz_to_rgb(xyz)
    M = inv(get_M_RGB2XYZ()); [r, c, ch] = size(xyz);
    lin = reshape(reshape(xyz, [], ch) * M', r, c, ch);
    rgb = min(max(linear_to_srgb(lin), 0.0), 1.0);
end

function val = f_func(t)
    delta = 6 / 29; val = zeros(size(t)); mask = t > delta^3;
    val(mask) = nthroot(t(mask), 3); val(~mask) = t(~mask) / (3 * delta^2) + 4 / 29;
end

function val = finv_func(t)
    delta = 6 / 29; val = zeros(size(t)); mask = t > delta;
    val(mask) = t(mask) .^ 3; val(~mask) = 3 * delta^2 * (t(~mask) - 4 / 29);
end

function lab = rgb_to_lab(rgb)
    white = reshape([0.95047, 1.00000, 1.08883], 1, 1, 3);
    xyz = rgb_to_xyz(rgb) ./ white;
    fx = f_func(xyz(:,:,1)); fy = f_func(xyz(:,:,2)); fz = f_func(xyz(:,:,3));
    lab = cat(3, 116 * fy - 16, 500 * (fx - fy), 200 * (fy - fz));
end

function rgb = lab_to_rgb(lab)
    white = reshape([0.95047, 1.00000, 1.08883], 1, 1, 3);
    fy = (lab(:,:,1) + 16) / 116; fx = fy + lab(:,:,2) / 500; fz = fy - lab(:,:,3) / 200;
    xyz = cat(3, finv_func(fx), finv_func(fy), finv_func(fz)) .* white;
```

```
    rgb = xyz_to_rgb(xyz);
end
```

Phase 1 – Part 2

```
% ===== Step 1: Select Main Folder Interactively =====
baseFolder = uigetdir('', 'Select the ductal_carcinoma or BreaKHis folder');

if baseFolder == 0
    error('No folder was selected!');
end

% Search recursively for all PNG images
allFiles = dir(fullfile(baseFolder, '**', '*.png'));
numImages = length(allFiles);

if numImages == 0
    error('No images found in the selected folder!');
else
    fprintf('Found %d images across patients. Processing statistics in Lab space...\n', numImages);
end

meanLAB = zeros(numImages, 3);
patientIDs = cell(numImages, 1);

% ===== Step 2: Compute Per-Image Statistics in L*a*b* Space =====
for i = 1:numImages
    imgPath = fullfile(allFiles(i).folder, allFiles(i).name);
    img = imread(imgPath);

    % Convert the image from RGB to CIE L*a*b* color space
    imgLab = rgb2lab(img);

    % Compute the mean value for each channel (L*, a*, b*)
    for c = 1:3
        channelData = imgLab(:,:,c);
        meanLAB(i, c) = mean(channelData(:));
    end

    patientIDs{i} = allFiles(i).name(1:min(18, length(allFiles(i).name)));
end

% ===== Step 3: Visualization (Boxplot in L*a*b* Space) =====
figure('Name', 'Inter-Patient Stain Variation (L*a*b*)', 'Color', 'w');

boxplot(meanLAB, {'L* (Lightness)', 'a* (Green-Red)', 'b* (Blue-Yellow)'}, ...
    'Colors', [0.2 0.2 0.2; 0.8 0.2 0.5; 0.2 0.4 0.8], 'Symbol', 'r+');

grid on;
ylabel('Mean Value (L*a*b* Scale)');
title('Quantification of Inter-Patient Stain Variation in CIE L*a*b* Space at 100x Magnification');

% ===== Step 4: Display Std-Dev Values for Table 3.1 =====
std_L = std(meanLAB(:,1));
std_a = std(meanLAB(:,2));
std_b = std(meanLAB(:,3));

fprintf('\n=============================================\n');
fprintf('Inter-Patient Std-Dev (Before Normalization):\n');
fprintf('L* Channel Std: %.3f\n', std_L);
```

```
fprintf('a* Channel Std: %.3f\n', std_a);
fprintf('b* Channel Std: %.3f\n', std_b);
fprintf('================================================\n');
```

Phase 1 – Part 3

```
% ===== Step 1: Select Main Folder Interactively =====
baseFolder = uigetdir('', 'Select the ductal_carcinoma or BreaKHis folder');

if baseFolder == 0
    error('No folder was selected!');
end

% Search recursively for all PNG images
allFiles = dir(fullfile(baseFolder, '**', '*.png'));
numImages = length(allFiles);

if numImages == 0
    error('No images found in the selected folder!');
else
    fprintf('Found %d images across patients. Processing statistics in Lab space...\n', numImages);
end

meanLAB = zeros(numImages, 3);
patientIDs = cell(numImages, 1);

% ===== Step 2: Compute Per-Image Statistics in L*a*b* Space =====
for i = 1:numImages
    imgPath = fullfile(allFiles(i).folder, allFiles(i).name);
    img = imread(imgPath);

    % Convert the image from RGB to CIE L*a*b* color space
    imgLab = rgb2lab(img);

    % Compute the mean value for each channel (L*, a*, b*)
    for c = 1:3
        channelData = imgLab(:,:,c);
        meanLAB(i, c) = mean(channelData(:));
    end

    patientIDs{i} = allFiles(i).name(1:min(18, length(allFiles(i).name)));
end

% ===== Step 3: Visualization (Boxplot in L*a*b* Space) =====
figure('Name', 'Inter-Patient Stain Variation (L*a*b*)', 'Color', 'w');

boxplot(meanLAB, {'L* (Lightness)', 'a* (Green-Red)', 'b* (Blue-Yellow)'}, ...
    'Colors', [0.2 0.2 0.2; 0.8 0.2 0.5; 0.2 0.4 0.8], 'Symbol', 'r+');

grid on;
ylabel('Mean Value (L*a*b* Scale)');
title('Quantification of Inter-Patient Stain Variation in CIE L*a*b* Space at 100x Magnification');

% ===== Step 4: Display Std-Dev Values for Table 3.1 =====
std_L = std(meanLAB(:,1));
std_a = std(meanLAB(:,2));
std_b = std(meanLAB(:,3));

fprintf('\n==========================================\n');
fprintf('Inter-Patient Std-Dev (Before Normalization):\n');
fprintf('L* Channel Std: %.3f\n', std_L);
```

```
fprintf('a* Channel Std: %.3f\n', std_a);
fprintf('b* Channel Std: %.3f\n', std_b);
fprintf('================================================\n');
```

Phase 1 – Part 4

```
% ===== Step 1: Read and Display Original Image =====
[fileName, pathName] = uigetfile({'*.png;*.jpg;*.tif', 'Image Files'}, 'Select Normalised H&E Image');
if isequal(fileName,0), error('No image selected!'); end
img = imread(fullfile(pathName, fileName));

% ===== Step 2: Method 1 - Colour Distance-Based Segmentation (DIP 4e Sec 6.7) =====
% Convert the image to double precision for processing
imgD = double(img);

% Define a reference color sample for the nuclei (Hematoxylin Nuclei Sample - Dark Violet)
% Reference RGB value approximating the nuclei color in H&E images:
sample_RGB = [80, 50, 110]; % Sample value for the nuclei color in H&E images

% Compute the Euclidean distance of each pixel from the reference color
distMap = sqrt((imgD(:,:,1) - sample_RGB(1)).^2 + ...
               (imgD(:,:,2) - sample_RGB(2)).^2 + ...
               (imgD(:,:,3) - sample_RGB(3)).^2);

% Determine a threshold for the distance map using Otsu's method
maxDist = max(distMap(:));
distNorm = distMap / maxDist;
levelDist = graythresh(distNorm);
seg_color = distNorm < (levelDist * 0.8); % Select pixels closer to the reference color than the threshold

% Remove small noise objects
seg_color = bwareaopen(seg_color, 20); % Remove small noise objects

% ===== Step 3: Method 2 - Single Grey Channel Segmentation =====
% Use the R channel of the image, as it provides good contrast for the nuclei
greyChan = img(:,:,1);
levelGrey = graythresh(greyChan);
seg_grey = greyChan < levelGrey; % Select pixels darker than the threshold

% Remove small noise objects
seg_grey = bwareaopen(seg_grey, 20);

% ===== Step 4: Visualization & Comparison =====
figure('Name', 'Nuclei Segmentation Comparison', 'Color', 'w');

subplot(2, 2, 1);
imshow(img);
title('Original H&E Image');

subplot(2, 2, 2);
imshow(distNorm, []);
title('Colour Distance Map (DIP 6.7)');
colormap(gca, 'jet'); colorbar;

subplot(2, 2, 3);
imshow(seg_color);
title('Colour-Based Segmentation Mask');

subplot(2, 2, 4);
imshow(seg_grey);
```

```
title('Single Grey Channel (R) Mask');
```

Phase 1 – Part 5

```
% ====== Step 1: Read Original Normalized Image ======
[fileName, pathName] = uigetfile({'*.png;*.jpg;*.tif', 'Image Files'}, 'Select Normalised H&E Image');
if isequal(fileName,0), error('No image selected!'); end
img = imread(fullfile(pathName, fileName));
greyImg = rgb2gray(img); % Convert to a grayscale image

% ====== Method 1: Intensity Slicing (DIP 4e Sec 6.3) ======
% Divide the gray levels into 4 ranges, then assign a different color to each
slicedImg = zeros([size(greyImg), 3], 'uint8');
L1 = greyImg < 64;                % Very dark region (shadows) -> Blue
L2 = greyImg >= 64 & greyImg < 128; % Medium-dark region (midtones) -> Green
L3 = greyImg >= 128 & greyImg < 192;% Medium-bright region -> Yellow
L4 = greyImg >= 192;              % Bright region -> Red

slicedImg(:,:,1) = (L3*255) + (L4*255);      % Red Channel
slicedImg(:,:,2) = (L2*255) + (L3*255);      % Green Channel
slicedImg(:,:,3) = (L1*255);                 % Blue Channel

% ====== Method 2: Grey-to-Colour Transformation (Continuous Colormap) ======
% Apply a continuous pseudo-color mapping using a colormap (Jet/Rainbow)
pseudoImg = ind2rgb(greyImg, jet(256));

% ====== Display Results ======
figure('Name', 'Pseudocolour Mapping Methods', 'Color', 'w');

subplot(1, 3, 1);
imshow(greyImg);
title('Original Grayscale');

subplot(1, 3, 2);
imshow(slicedImg);
title('Intensity Slicing (Discrete)');

subplot(1, 3, 3);
imshow(pseudoImg);
title('Grey-to-Colour Transformation (Jet)');
```

Phase 2 – Part 1

```
% ===== Step 1: Read Image and Convert to Grayscale (From Scratch) =====
[fileName, pathName] = uigetfile({'*.png;*.jpg;*.tif', 'Image Files'}, 'Select Image');
if isequal(fileName,0), error('No image selected!'); end
imgRGB = imread(fullfile(pathName, fileName));

% Grey Conversion From Scratch: Y = 0.2989*R + 0.5870*G + 0.1140*B
img = uint8(0.2989*double(imgRGB(:,:,1)) + 0.5870*double(imgRGB(:,:,2)) + 0.1140*double(imgRGB(:,:,3)));
[M, N] = size(img);
numPixels = M * N;

% ===== 1. Coding Redundancy (From Scratch) =====
% Histogram computation from scratch
counts = zeros(1, 256);
for i = 1:M
    for j = 1:N
        val = img(i,j) + 1; % MATLAB 1-based indexing
        counts(val) = counts(val) + 1;
    end
end
p = counts / numPixels; % Probabilities

% Entropy calculation from scratch: H = - sum(p * log2(p))
H = 0;
for k = 1:256
    if p(k) > 0
        H = H - p(k) * log2(p(k));
    end
end

L_avg = 8; % Fixed 8-bit representation
coding_redundancy_bits = L_avg - H;
coding_redundancy_percent = (1 - (H / L_avg)) * 100;

% ===== 2. Spatial Redundancy (From Scratch) =====
% Horizontal spatial correlation between adjacent pixels x(i,j) and x(i, j+1)
x = double(img(:, 1:end-1));
y = double(img(:, 2:end));

mean_x = sum(x(:)) / numel(x);
mean_y = sum(y(:)) / numel(y);

num = sum((x(:) - mean_x) .* (y(:) - mean_y));
den = sqrt(sum((x(:) - mean_x).^2) * sum((y(:) - mean_y).^2));
r_spatial = num / den;

spatial_redundancy_percent = r_spatial * 100;

% ===== 3. Irrelevant Information (From Scratch) =====
% Quantization/Visual thresholding: Discarding sub-visual background noise (< 15 grey levels)
q_step = 16; % Quantization step
img_quantized = floor(double(img) / q_step) * q_step;

% Calculate percentage of redundant/discarded details in smooth background
diff_img = abs(double(img) - img_quantized);
irrelevant_pixels = sum(diff_img(:) < 8);
irrelevant_info_percent = (irrelevant_pixels / numPixels) * 100;
```

```
% ===== Print Actual Measured Numbers =====
fprintf('\n================ MEASURED REDUNDANCIES ================\n');
fprintf('1. Coding Redundancy:\n');
fprintf('   - Image Entropy (H): %.4f bits/pixel\n', H);
fprintf('   - Average Bit Length (L_avg): %d bits/pixel\n', L_avg);
fprintf('   - Redundant Coding Bits: %.4f bits/pixel (%.2f%%)\n\n', coding_redundancy_bits, coding_redundancy_percent);

fprintf('2. Spatial Redundancy:\n');
fprintf('   - Adjacent Pixel Correlation (r): %.4f\n', r_spatial);
fprintf('   - Spatial Redundancy Measure: %.2f%%\n\n', spatial_redundancy_percent);

fprintf('3. Irrelevant Information:\n');
fprintf('   - Psychovisual Redundancy (Non-essential Background): %.2f%%\n', irrelevant_info_percent);
fprintf('=======================================================\n');
```

Phase 2 – Part 2

```
% ====== Step 1: Read Image & Convert to Grayscale ======
[fileName, pathName] = uigetfile({'*.png;*.jpg;*.tif', 'Image Files'}, 'Select Image');
if isequal(fileName,0), error('No image selected!'); end
imgRGB = imread(fullfile(pathName, fileName));

% Grayscale conversion from scratch
img = uint8(0.2989*double(imgRGB(:,:,1)) + 0.5870*double(imgRGB(:,:,2)) + 0.1140*double(imgRGB(:,:,3)));
[M, N] = size(img);
totalPixels = M * N;
originalBits = totalPixels * 8; % 8 bits per pixel baseline

% ===== 1. Huffman Coding (From Scratch Calculation) =====
% Calculate histogram & probabilities
counts = zeros(1, 256);
for i = 1:totalPixels
    val = img(i) + 1;
    counts(val) = counts(val) + 1;
end
p = counts / totalPixels;
valid_p = p(p > 0);

% Build Huffman Tree and generate code lengths recursively/iteratively
symbols = num2cell(1:length(valid_p));
probs = valid_p;
codeLens = zeros(size(valid_p));

% Tracking nodes for tree depth calculation
nodeTree = cell(size(valid_p));
for i = 1:length(valid_p)
    nodeTree{i} = i;
end

while length(probs) > 1
    % Sort probabilities ascending
    [probs, idx] = sort(probs, 'ascend');
    nodeTree = nodeTree(idx);

    % Merge two lowest probabilities
    mergedProb = probs(1) + probs(2);

    % Update code length for children in merged nodes
    for k = 1:length(nodeTree{1})
        codeLens(nodeTree{1}(k)) = codeLens(nodeTree{1}(k)) + 1;
    end
    for k = 1:length(nodeTree{2})
        codeLens(nodeTree{2}(k)) = codeLens(nodeTree{2}(k)) + 1;
    end

    % Combine nodes
    newNode = [nodeTree{1}, nodeTree{2}];
    probs = [mergedProb, probs(3:end)];
    nodeTree = [ {newNode}, nodeTree(3:end) ];
end

% Compute Huffman Bits
L_huffman = sum(valid_p .* codeLens);
huffmanTotalBits = L_huffman * totalPixels;
CR_huffman = originalBits / huffmanTotalBits;
```

```
% ===== 2. Run-Length Coding (RLC From Scratch) =====
% Vectorize image
pixelVec = img(:);
runLengths = [];
runValues = [];

currentVal = pixelVec(1);
currentCount = 1;

for i = 2:totalPixels
    if pixelVec(i) == currentVal && currentCount < 255
        currentCount = currentCount + 1;
    else
        runLengths(end+1) = currentCount;
        runValues(end+1) = currentVal;
        currentVal = pixelVec(i);
        currentCount = 1;
    end
end
runLengths(end+1) = currentCount;
runValues(end+1) = currentVal;

% Total bits for RLC (Each run takes 1 byte for length + 1 byte for value = 16 bits)
rlcTotalBits = length(runLengths) * 16;
CR_rlc = originalBits / rlcTotalBits;

% ===== Display Measured Numbers =====
fprintf('\n================= COMPRESSION RESULTS ================\n');
fprintf('Original Image Size: %d x %d pixels (%d bits)\n\n', M, N, originalBits);
fprintf('1. Huffman Coding Alone:\n');
fprintf('   - Average Code Length (L_avg): %.4f bits/pixel\n', L_huffman);
fprintf('   - Total Encoded Bits: %.0f bits\n', huffmanTotalBits);
fprintf('   - Compression Ratio (CR_Huffman): %.2f:1\n', CR_huffman);
fprintf('   - Bit Saving: %.2f%%\n\n', (1 - 1/CR_huffman)*100);

fprintf('2. Run-Length Coding (RLC) Alone:\n');
fprintf('   - Number of Runs: %d\n', length(runLengths));
fprintf('   - Total Encoded Bits: %.0f bits\n', rlcTotalBits);
fprintf('   - Compression Ratio (CR_RLC): %.2f:1\n', CR_rlc);
fprintf('=======================================================\n');
```

Phase 2 – Part 3

```
% ========================================================================
% BLOCK-DCT (JPEG-LIKE) CODEC WITH QUALITY FACTOR SWEEP (FROM SCRATCH)
% ========================================================================

% ====== Step 1: Read Image & Convert to Grayscale ======
[fileName, pathName] = uigetfile({'*.png;*.jpg;*.tif', 'Image Files'}, 'Select Image');
if isequal(fileName,0), error('No image selected!'); end
imgRGB = imread(fullfile(pathName, fileName));

% Grayscale conversion from scratch
img = uint8(0.2989*double(imgRGB(:,:,1)) + 0.5870*double(imgRGB(:,:,2)) + 0.1140*double(imgRGB(:,:,3)));
[M, N] = size(img);

% Crop dimensions to multiples of 8
M8 = floor(M/8) * 8;
N8 = floor(N/8) * 8;
imgC = double(img(1:M8, 1:N8)) - 128;
orig_cropped = uint8(imgC + 128);

% Standard JPEG Base Luminance Matrix
Q_base = [
   16 11 10 16 24 40 51 61;
   12 12 14 19 26 58 60 55;
   14 13 16 24 40 57 69 56;
   14 17 22 29 51 87 80 62;
   18 22 37 56 68 109 103 77;
   24 35 55 64 81 104 113 92;
   49 64 78 87 103 121 120 101;
   72 92 95 98 112 100 103 99
];

% ====== Step 2: Compute DCT Orthonormal Basis Matrix C (From Scratch) ======
C = zeros(8,8);
for i = 0:7
   for j = 0:7
      if i == 0
         alpha = sqrt(1/8);
      else
         alpha = sqrt(2/8);
      end
      C(i+1, j+1) = alpha * cos((2*j + 1) * i * pi / 16);
   end
end

% ====== Step 3: Single Quality Reconstruction (Q = 50) for Visual Display ======
quality_default = 50;
if quality_default < 50
   S_def = 5000 / quality_default;
else
   S_def = 200 - 2 * quality_default;
end
Q_scaled_def = max(1, floor((Q_base * S_def + 50) / 100));

dct_quant_def = zeros(M8, N8);
for r = 1:8:M8
   for c = 1:8:N8
      block = imgC(r:r+7, c:c+7);
```

```
        dct_block = C * block * C';
        dct_quant_def(r:r+7, c:c+7) = round(dct_block ./ Q_scaled_def);
    end
end

img_rec_def = zeros(M8, N8);
for r = 1:8:M8
    for c = 1:8:N8
        q_block = dct_quant_def(r:r+7, c:c+7);
        img_rec_def(r:r+7, c:c+7) = C' * (q_block .* Q_scaled_def) * C;
    end
end
img_rec_def = uint8(min(max(img_rec_def + 128, 0), 255));

diff_def = double(orig_cropped) - double(img_rec_def);
MSE_def = sum(diff_def(:).^2) / (M8 * N8);
PSNR_def = 10 * log10((255^2) / MSE_def);

% Plot Visual Comparison
figure('Name', 'Block-DCT Reconstruction (Q=50)', 'Color', 'w');
subplot(1, 2, 1); imshow(orig_cropped); title('Original Grayscale');
subplot(1, 2, 2); imshow(img_rec_def); title(sprintf('Reconstructed (Quality=50, PSNR=%.2fdB)', PSNR_def));

% ===== Step 4: Quality Factor Sweep (Q = 5 to 90) =====
Q_list = [5, 10, 20, 30, 50, 70, 90];
results = zeros(length(Q_list), 4); % Columns: Q, bpp, PSNR, SSIM

for q_idx = 1:length(Q_list)
    quality = Q_list(q_idx);

    if quality < 50
        S = 5000 / quality;
    else
        S = 200 - 2 * quality;
    end
    Q_scaled = max(1, floor((Q_base * S + 50) / 100));

    dct_quant = zeros(M8, N8);
    for r = 1:8:M8
        for c = 1:8:N8
            block = imgC(r:r+7, c:c+7);
            dct_block = C * block * C';
            dct_quant(r:r+7, c:c+7) = round(dct_block ./ Q_scaled);
        end
    end

    % Decoder Reconstruction
    img_rec = zeros(M8, N8);
    for r = 1:8:M8
        for c = 1:8:N8
            q_block = dct_quant(r:r+7, c:c+7);
            img_rec(r:r+7, c:c+7) = C' * (q_block .* Q_scaled) * C;
        end
    end
    img_rec = uint8(min(max(img_rec + 128, 0), 255));

    % 1. Estimated Bits per Pixel (bpp)
    non_zero = sum(dct_quant(:) ~= 0);
    bpp = (non_zero * 4.6) / (M8 * N8); % Approximated bits per pixel from nonzero coefficients
```

```
    % 2. MSE & PSNR Calculation
    diff = double(orig_cropped) - double(img_rec);
    MSE = sum(diff(:).^2) / (M8 * N8);
    PSNR = 10 * log10((255^2) / MSE);

    % 3. SSIM Calculation (From Scratch)
    mu_x = mean(double(orig_cropped(:)));
    mu_y = mean(double(img_rec(:)));
    sig_x = var(double(orig_cropped(:)));
    sig_y = var(double(img_rec(:)));
    sig_xy = cov(double(orig_cropped(:)), double(img_rec(:)));
    sig_xy = sig_xy(1,2);

    c1 = (0.01 * 255)^2; c2 = (0.03 * 255)^2;
    ssim_val = ((2*mu_x*mu_y + c1)*(2*sig_xy + c2)) / ((mu_x^2 + mu_y^2 + c1)*(sig_x + sig_y + c2));

    results(q_idx, :) = [quality, bpp, PSNR, ssim_val];
end

% ===== Step 5: Print Results Table =====
fprintf('\n==================== QUALITY SWEEP RESULTS ====================\n');
fprintf('Quality (Q)\tbits/pixel (bpp)\tPSNR (dB)\tSSIM\n');
fprintf('---------------------------------------------------------------\n');
for i = 1:size(results,1)
    fprintf('%d\t\t%.3f\t\t\t%.2f\t\t%.3f\n', results(i,1), results(i,2), results(i,3), results(i,4));
end
fprintf('===============================================================\n');
```

Phase 2 – Part 4

```
% ====== Step 1: Read Image & Prepare Grayscale ======
[fileName, pathName] = uigetfile({'*.png;*.jpg;*.tif', 'Image Files'}, 'Select Image');
if isequal(fileName,0), error('No image selected!'); end
imgRGB = imread(fullfile(pathName, fileName));

% Grayscale conversion
img = uint8(0.2989*double(imgRGB(:,:,1)) + 0.5870*double(imgRGB(:,:,2)) + 0.1140*double(imgRGB(:,:,3)));
[M, N] = size(img);
M8 = floor(M/8) * 8; N8 = floor(N/8) * 8;
imgC = double(img(1:M8, 1:N8));

% ====== Step 2: DCT Sweep Data ======
Q_list = [5, 10, 20, 30, 50, 70, 90];
dct_bpp = [0.170, 0.313, 0.493, 0.616, 0.792, 0.996, 1.517];
dct_psnr = [25.64, 29.26, 32.65, 34.47, 36.56, 38.41, 42.11];

% ====== Step 3: Wavelet (DWT bior4.4 / Haar) Quality Sweep ======
wav_results = zeros(length(Q_list), 4); % Q, bpp, PSNR, SSIM

% DWT Level 3 Decomposition
[C_wav, L_wav] = wavedec2(imgC, 3, 'bior4.4');

for q_idx = 1:length(Q_list)
    quality = Q_list(q_idx);

    % Scalable Quantization threshold based on quality
    thresh = 200 / quality;
    C_quant = round(C_wav / thresh) * thresh;

    % Reconstruction via IDWT
    img_rec_wav = waverec2(C_quant, L_wav, 'bior4.4');
    img_rec_wav = uint8(min(max(img_rec_wav, 0), 255));

    % 1. Bits per pixel (non-zero coefficient estimation)
    non_zero = sum(C_quant ~= 0);
    bpp_wav = (non_zero * 4.2) / (M8 * N8);

    % 2. PSNR
    diff_wav = double(imgC) - double(img_rec_wav);
    MSE_wav = mean(diff_wav(:).^2);
    psnr_wav = 10 * log10((255^2) / MSE_wav);

    % 3. SSIM
    mu_x = mean(imgC(:)); mu_y = mean(double(img_rec_wav(:)));
    sig_x = var(imgC(:)); sig_y = var(double(img_rec_wav(:)));
    sig_xy = cov(imgC(:), double(img_rec_wav(:))); sig_xy = sig_xy(1,2);
    c1 = (0.01 * 255)^2; c2 = (0.03 * 255)^2;
    ssim_wav = ((2*mu_x*mu_y + c1)*(2*sig_xy + c2)) / ((mu_x^2 + mu_y^2 + c1)*(sig_x + sig_y + c2));

    wav_results(q_idx, :) = [quality, bpp_wav, psnr_wav, ssim_wav];
end

% ====== Step 4: Print Wavelet Table ======
fprintf('\n================= WAVELET SWEEP RESULTS ==================\n');
fprintf('Quality (Q)\tbits/pixel (bpp)\tPSNR (dB)\tSSIM\n');
fprintf('-----------------------------------------------------------\n');
for i = 1:size(wav_results,1)
```

```
    fprintf('%d\t\t%.3f\t\t\t%.2f\t\t%.3f\n', wav_results(i,1), wav_results(i,2), wav_results(i,3), wav_results(i,4));
end
fprintf('=================================================================\n');

% ===== Step 5: Plot Rate-Distortion Curve (DCT vs Wavelet) =====
figure('Name', 'Rate-Distortion: DCT vs Wavelet', 'Color', 'w');
plot(dct_bpp, dct_psnr, '-o', 'LineWidth', 2, 'MarkerSize', 6, 'DisplayName', 'Block-DCT (ours)');
hold on;
plot(wav_results(:,2), wav_results(:,3), '-s', 'LineWidth', 2, 'MarkerSize', 6, 'DisplayName', 'Wavelet (pywt/DWT)');
grid on;
xlabel('bits per pixel (bpp)');
ylabel('PSNR (dB)');
title('Rate-distortion: DCT vs Wavelet');
legend('Location', 'southeast');
```

Phase 2 – Part 5

```
% ===== Step 1: Read Color Image =====
[fileName, pathName] = uigetfile({'*.png;*.jpg;*.tif', 'Image Files'}, 'Select Color Image');
if isequal(fileName,0), error('No image selected!'); end
imgRGB = imread(fullfile(pathName, fileName));

[M, N, ~] = size(imgRGB);
M8 = floor(M/16) * 16; N8 = floor(N/16) * 16;
imgRGB = imgRGB(1:M8, 1:N8, :);

% Base Matrix & Basis DCT
Q_base = [16 11 10 16 24 40 51 61; 12 12 14 19 26 58 60 55; 14 13 16 24 40 57 69 56;
          14 17 22 29 51 87 80 62; 18 22 37 56 68 109 103 77; 24 35 55 64 81 104 113 92;
          49 64 78 87 103 121 120 101; 72 92 95 98 112 100 103 99];

C_mat = zeros(8,8);
for i = 0:7, for j = 0:7
    if i == 0, alpha = sqrt(1/8); else, alpha = sqrt(2/8); end
    C_mat(i+1, j+1) = alpha * cos((2*j + 1) * i * pi / 16);
end, end

Q_levels = [20, 50, 80];
table_data = zeros(3, 6); % Q, RGB_bits, RGB_PSNR, YCbCr_bits, YCbCr_PSNR, Bit_Saving

for idx = 1:length(Q_levels)
    q_val = Q_levels(idx);

    % Scale Matrix
    if q_val < 50, S = 5000 / q_val; else, S = 200 - 2 * q_val; end
    Q_scaled = max(1, floor((Q_base * S + 50) / 100));

    % --- 1. RGB Independent ---
    rec_RGB = zeros(M8, N8, 3); nz_rgb = 0;
    for c = 1:3
        ch = double(imgRGB(:,:,c)) - 128;
        q_ch = zeros(M8, N8); rec_ch = zeros(M8, N8);
        for r = 1:8:M8, for col = 1:8:N8
            blk = ch(r:r+7, col:col+7);
            q_blk = round((C_mat * blk * C_mat') ./ Q_scaled);
            q_ch(r:r+7, col:col+7) = q_blk;
            rec_ch(r:r+7, col:col+7) = (C_mat' * (q_blk .* Q_scaled) * C_mat) + 128;
        end, end
        rec_RGB(:,:,c) = rec_ch;
        nz_rgb = nz_rgb + sum(q_ch(:) ~= 0);
    end
    bits_rgb = round(nz_rgb * 4.6);
    mse_rgb = mean((double(imgRGB(:)) - double(rec_RGB(:))).^2);
    psnr_rgb = 10 * log10((255^2) / mse_rgb);

    % --- 2. YCbCr 4:2:0 Subsampled ---
    imgYCBCR = rgb2ycbcr(imgRGB);
    Y = double(imgYCBCR(:,:,1)) - 128;
    Cb_sub = imresize(double(imgYCBCR(:,:,2)), 0.5, 'bilinear') - 128;
    Cr_sub = imresize(double(imgYCBCR(:,:,3)), 0.5, 'bilinear') - 128;
    [M_sub, N_sub] = size(Cb_sub);

    % Quantize Y
```

```
    q_Y = zeros(M8, N8); rec_Y = zeros(M8, N8);
    for r = 1:8:M8, for col = 1:8:N8
        blk = Y(r:r+7, col:col+7);
        q_blk = round((C_mat * blk * C_mat') ./ Q_scaled);
        q_Y(r:r+7, col:col+7) = q_blk;
        rec_Y(r:r+7, col:col+7) = (C_mat' * (q_blk .* Q_scaled) * C_mat) + 128;
    end, end

    % Quantize Cb, Cr
    Q_chroma = max(1, floor((Q_scaled * 1.5)));
    q_Cb = zeros(M_sub, N_sub); rec_Cb_sub = zeros(M_sub, N_sub);
    q_Cr = zeros(M_sub, N_sub); rec_Cr_sub = zeros(M_sub, N_sub);
    for r = 1:8:M_sub, for col = 1:8:N_sub
        q_cb = round((C_mat * Cb_sub(r:r+7, col:col+7) * C_mat') ./ Q_chroma);
        q_cr = round((C_mat * Cr_sub(r:r+7, col:col+7) * C_mat') ./ Q_chroma);
        q_Cb(r:r+7, col:col+7) = q_cb; q_Cr(r:r+7, col:col+7) = q_cr;
        rec_Cb_sub(r:r+7, col:col+7) = (C_mat' * (q_cb .* Q_chroma) * C_mat) + 128;
        rec_Cr_sub(r:r+7, col:col+7) = (C_mat' * (q_cr .* Q_chroma) * C_mat) + 128;
    end, end

    rec_Cb = imresize(rec_Cb_sub, [M8 N8], 'bilinear');
    rec_Cr = imresize(rec_Cr_sub, [M8 N8], 'bilinear');
    rec_YCBCR = ycbcr2rgb(uint8(cat(3, min(max(rec_Y,0),255), min(max(rec_Cb,0),255), min(max(rec_Cr,0),255))));

    bits_ycbcr = round((sum(q_Y(:)~=0) + sum(q_Cb(:)~=0) + sum(q_Cr(:)~=0)) * 4.6);
    mse_ycbcr = mean((double(imgRGB(:)) - double(rec_YCBCR(:))).^2);
    psnr_ycbcr = 10 * log10((255^2) / mse_ycbcr);

    saving = ((bits_rgb - bits_ycbcr) / bits_rgb) * 100;
    table_data(idx, :) = [q_val, bits_rgb, psnr_rgb, bits_ycbcr, psnr_ycbcr, saving];
end

% ===== Step 2: Print Table =====
fprintf('\n================= COLOR COMPRESSION TABLE ==================\n');
fprintf('Quality\tRGB bits\tRGB PSNR\tYCbCr bits\tYCbCr PSNR\tBit saving\n');
fprintf('-------------------------------------------------------------\n');
for i = 1:3
    fprintf('%d\t%d\t%.2f\t\t%d\t\t%.2f\t\t%.1f%%\n', table_data(i,1), table_data(i,2), table_data(i,3), table_data(i,4),
table_data(i,5), table_data(i,6));
end
fprintf('=============================================================\n');

% ===== Step 3: Bar Chart Comparison =====
figure('Name', 'RGB vs YCbCr Size', 'Color', 'w');
bar_data = [table_data(:,2)/1000, table_data(:,4)/1000]; % convert to kbits
b = bar(bar_data);
b(1).FaceColor = [0.12, 0.47, 0.71]; % Blue
b(2).FaceColor = [1.00, 0.50, 0.05]; % Orange
set(gca, 'XTickLabel', {'Q=20', 'Q=50', 'Q=80'});
ylabel('kbits');
title('RGB-independent vs YCbCr 4:2:0 compressed size');
legend({'RGB-independent', 'YCbCr 4:2:0'}, 'Location', 'northwest');
grid on;
```

Phase 3 – Part 1

No code for this part.

Phase 2 – Part 2

```
% ====== Step 1: Read Original Image ======
[fileName, pathName] = uigetfile({'*.png;*.jpg;*.tif', 'Image Files'}, 'Select Image');
if isequal(fileName,0), error('No image selected!'); end
imgRGB = imread(fullfile(pathName, fileName));

% Basic Segmentation Function (Thresholding / Otsu on Hematoxylin / Grayscale)
segment_nuclei = @(I) imbinarize(I, graythresh(I));

% Ground Truth Segmentation on Original
imgGray = rgb2gray(imgRGB);
bw_orig = ~segment_nuclei(imgGray);
bw_orig = bwareaopen(bw_orig, 15); % Filter noise

stats_orig = regionprops(bw_orig, 'Area');
orig_count = length(stats_orig);
orig_total_area = sum([stats_orig.Area]);

% ====== Step 2: Quality Factor Sweep & Downstream Segmentation ======
Q_levels = [5, 10, 20, 30, 50, 70, 90];
results_downstream = zeros(length(Q_levels), 6); % Q, bpp, PSNR, Count Error %, Area Error %, Nuclei Count

for i = 1:length(Q_levels)
    Q = Q_levels(i);

    % Simulate DCT Reconstruction (or load reconstructed images)
    % Here using JPEG compression simulation
    imwrite(imgRGB, 'temp.jpg', 'Quality', Q);
    img_rec = imread('temp.jpg');

    % Segment Reconstructed Image
    img_rec_gray = rgb2gray(img_rec);
    bw_rec = ~segment_nuclei(img_rec_gray);
    bw_rec = bwareaopen(bw_rec, 15);

    stats_rec = regionprops(bw_rec, 'Area');
    rec_count = length(stats_rec);
    rec_total_area = sum([stats_rec.Area]);

    % Errors
    count_err_pct = (abs(rec_count - orig_count) / orig_count) * 100;
    area_err_pct = (abs(rec_total_area - orig_total_area) / orig_total_area) * 100;

    % Bitrate and PSNR Calculation
    file_info = dir('temp.jpg');
    bpp = (file_info.bytes * 8) / (size(imgRGB,1) * size(imgRGB,2));
    mse = mean((double(imgRGB(:)) - double(img_rec(:))).^2);
    psnr_val = 10 * log10((255^2) / mse);

    results_downstream(i, :) = [Q, bpp, psnr_val, rec_count, count_err_pct, area_err_pct];
end
delete('temp.jpg');

% ====== Step 3: Print Results Table ======
fprintf('\n================= DOWNSTREAM TASK DEGRADATION TABLE ==================\n');
fprintf('Quality (Q)\tbits/pixel\tPSNR (dB)\tNuclei Count\tCount Error (%%)\tArea Error (%%)\n');
fprintf('----------------------------------------------------------------------\n');
for i = 1:length(Q_levels)
```

```
    fprintf('%d\t\t%.3f\t\t%.2f\t\t%d\t\t%.2f%%\t\t%.2f%%\n', ...
        results_downstream(i,1), results_downstream(i,2), results_downstream(i,3), ...
        results_downstream(i,4), results_downstream(i,5), results_downstream(i,6));
end
fprintf('================================================================================\n');

% ===== Step 4: Plot Downstream Error Curve =====
figure('Name', 'Downstream Task Degradation', 'Color', 'w');
plot(results_downstream(:,1), results_downstream(:,5), '-o', 'LineWidth', 2, 'DisplayName', 'Nuclei Count Error (%)');
hold on;
plot(results_downstream(:,1), results_downstream(:,6), '-s', 'LineWidth', 2, 'DisplayName', 'Total Nuclear Area Error (%)');
grid on;
xlabel('Quality Factor (Q)');
ylabel('Segmentation Error (%)');
title('Downstream Task Degradation vs Compression Quality');
legend('Location', 'northeast');
```

Phase 3 – Part 3

No code for this part.

Phase 3 – Part 4

```
% ====== Step 1: Read Image & Convert to Grayscale ======
[fileName, pathName] = uigetfile({'*.png;*.jpg;*.tif;*.bmp', 'Image Files'}, 'Select Image');
if isequal(fileName,0)
   img = uint8(randi([50 200], 256, 256)); % Fallback dummy image
else
   img = imread(fullfile(pathName, fileName));
end

if size(img,3) == 3, img = rgb2gray(img); end

% Ensure dimensions are exact multiples of 8
[H, W] = size(img);
H = floor(H/8) * 8;
W = floor(W/8) * 8;
img = img(1:H, 1:W);

% ====== Step 2: Fragile Watermark Insertion (LSB per 8x8 Block) ======
blockSize = 8;
numBlocksH = H / blockSize;
numBlocksW = W / blockSize;
watermarked_img = img;

% Generate Pseudo-Random Watermark Key Matrix
rng(2026); % Secret Key
watermark_key = uint8(randi([0 1], numBlocksH, numBlocksW));

for i = 1:numBlocksH
   for j = 1:numBlocksW
      r = (i-1)*8 + 1;
      c = (j-1)*8 + 1;

      % Embed 1-bit watermark into LSB of top-left pixel in each 8x8 block
      pixel_val = watermarked_img(r, c);
      bit_to_embed = watermark_key(i, j);
      watermarked_img(r, c) = bitset(pixel_val, 1, bit_to_embed);
   end
end

% ====== Step 3: Simulate Tampering (Modifying a Region) ======
tampered_img = watermarked_img;
center_y = round(H/2); center_x = round(W/2);
% Tamper a 40x40 region in the center (simulating cell structure alteration)
tampered_img(center_y-20:center_y+20, center_x-20:center_x+20) = 255;

% ====== Step 4: Fragile Watermark Extraction & Tamper Detection ======
tamper_map = zeros(numBlocksH, numBlocksW);

for i = 1:numBlocksH
   for j = 1:numBlocksW
      r = (i-1)*8 + 1;
      c = (j-1)*8 + 1;

      extracted_bit = bitget(tampered_img(r, c), 1);
      expected_bit = watermark_key(i, j);

      if extracted_bit ~= expected_bit
         tamper_map(i, j) = 1; % Mark block as tampered
```

```
        end
    end
end

% ===== Step 5: Plot Visual Results =====
figure('Name', 'Fragile Watermark Tamper Detection', 'Color', 'w');
subplot(1,3,1); imshow(watermarked_img); title('Watermarked Slide');
subplot(1,3,2); imshow(tampered_img); title('Tampered Slide');
subplot(1,3,3); imshow(tamper_map, []); title('Detected Tamper Map');
colormap(subplot(1,3,3), jet);
```